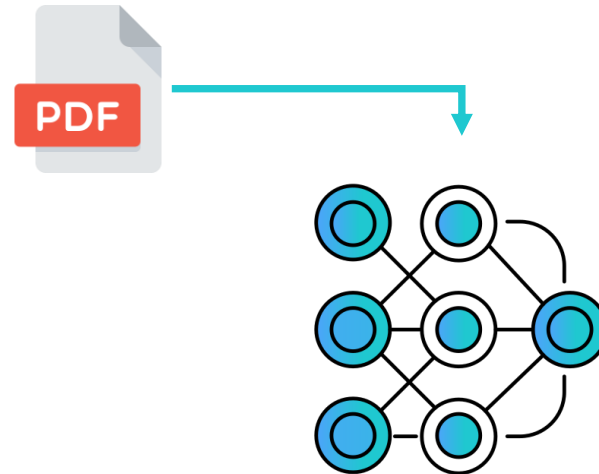
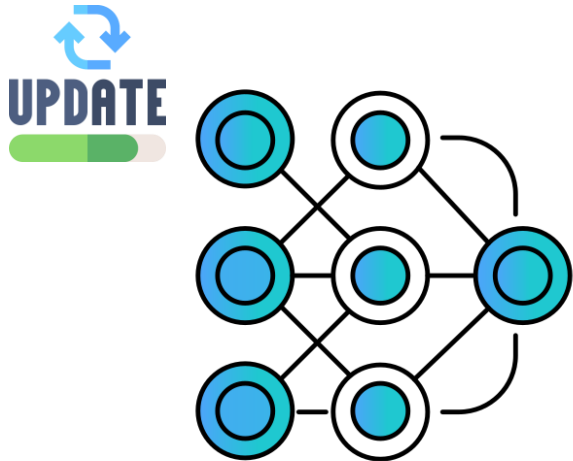


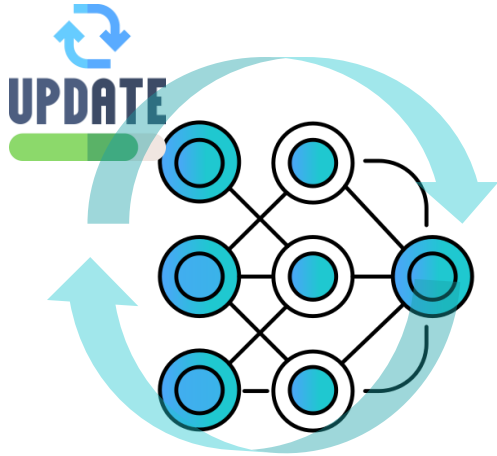
0. LLM 이전의 언어 모델

- 통계적 언어 모델 : “나는 _ 먹었다” 라는 문장에서, _에 올 단어를 ‘통계적으로’ 찾아내는 모델
 - “나는” 과 “먹었다” 와 함께 나타나는 단어들이 무엇일까? -> 가장 빈도수가 높은 “밥을” 을 찾음
- 신경망 언어 모델 : 데이터의 패턴을 학습하여 문제를 해결
 - 만족, 추천, 최고 <- 긍정적 표현이구나!
 - 별로, 비추, 실망 <- 부정적 표현이구나!
 - Ex) RNN, LSTM 등
- 트랜스포머가 탑재된 언어 모델 : 언어 모델이 ‘문장과 단락 전체’를 보고 단어에 집중할 수 있게 만듦
 - BERT, GPT
- 거대 언어 모델 : 파라미터가 15억 이상이 될 만큼, 대규모 데이터로 훈련된 인공지능 기반 언어모델.

1. LLM

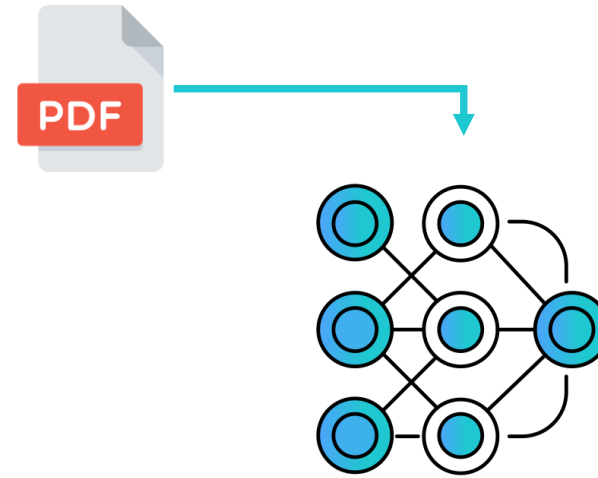
- LLM(Large Language Model) 이란?
 - 대규모 언어 모델. 방대한 양의 텍스트 데이터를 학습하여, 인간과 유사한 방식으로 자연어를 이해하고, 생성하는 인공지능 모델 by ChatGPT 4.0
- 우리는 LLM을
 - 1. 파인 튜닝(fine-tuning)하거나
 - 2. RAG(retrieval augmented generation)를 활용하여 사용할 수 있다.





파인 튜닝

- 장점
 - 특정 도메인에 대한 성능 향상
 - 특정 작업에 대해 정확하고 일관된 결과를 도출할 수 있음
- 단점
 - 새 도메인에 대한 재 학습 필요
 - 원 모델의 일반화 성능 저하

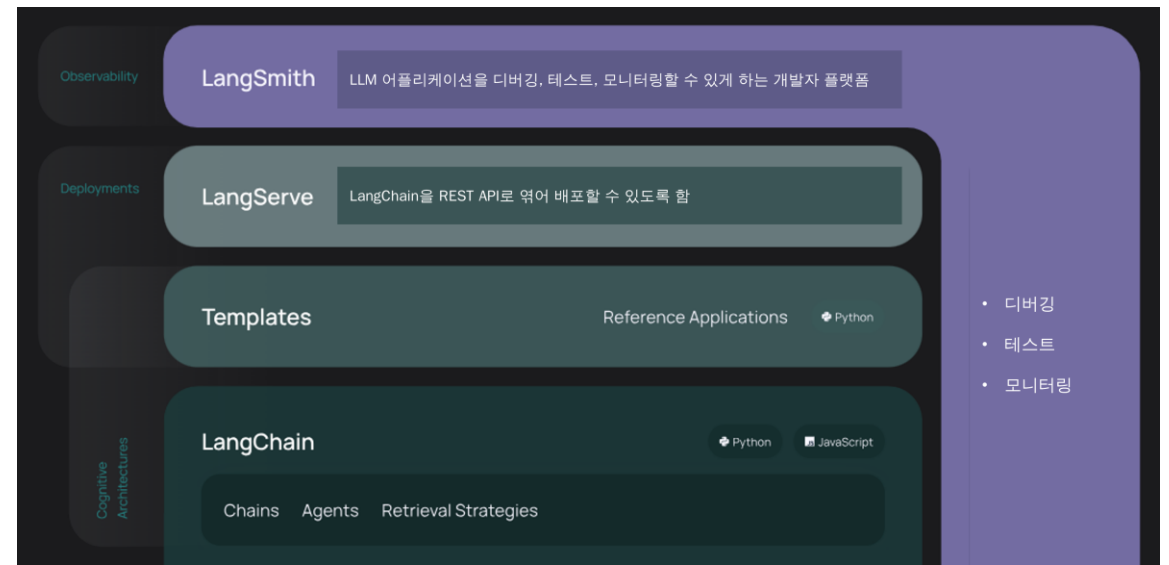


RAG

- 장점
 - 최신 정보의 습득으로 시의성과 정확성 상승
 - 다양한 도메인에 쉽게 적용 가능
 - 모델에 직접 학습시키지 않고도 사용 가능
- 단점
 - 검색 시스템의 품질이 영향을 미침
 - 검색과 생성 단계의 통합이 어려울 수 있음
 - 응답 속도가 느려질 수 있음(2stage)

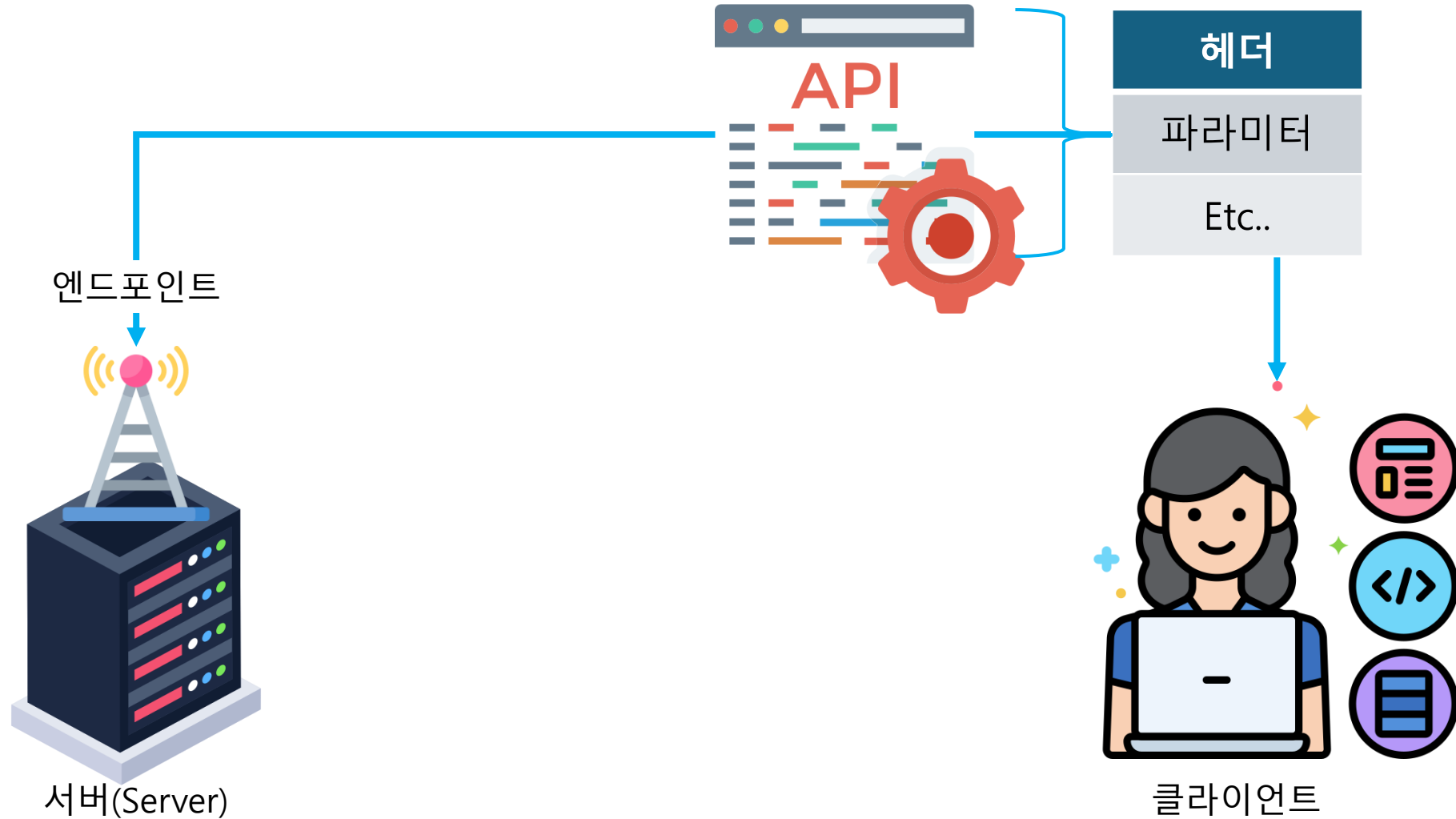
2. LangChain

- LangChain 이란?
 - LLM을 활용한 애플리케이션 생성, 평가, 배포 등을 편리하게 수행하도록 도와주는 라이브러리
- LangChain의 특징 :
 - 오픈 소스 라이브러리 : 모듈을 활용하여 애플리케이션을 구축하고, 수백 개의 LLM Provider와 통합
 - 상품화 지원 : LangSmith를 활용하여 앱을 테스트, 모니터링하여 최적화 및 배포
 - 배포 : LangServe를 사용하여 체인을 RestAPI로 변환, 보다 자유롭게 사용 가능



(참고자료) API 기본 개념 및 작동 원리

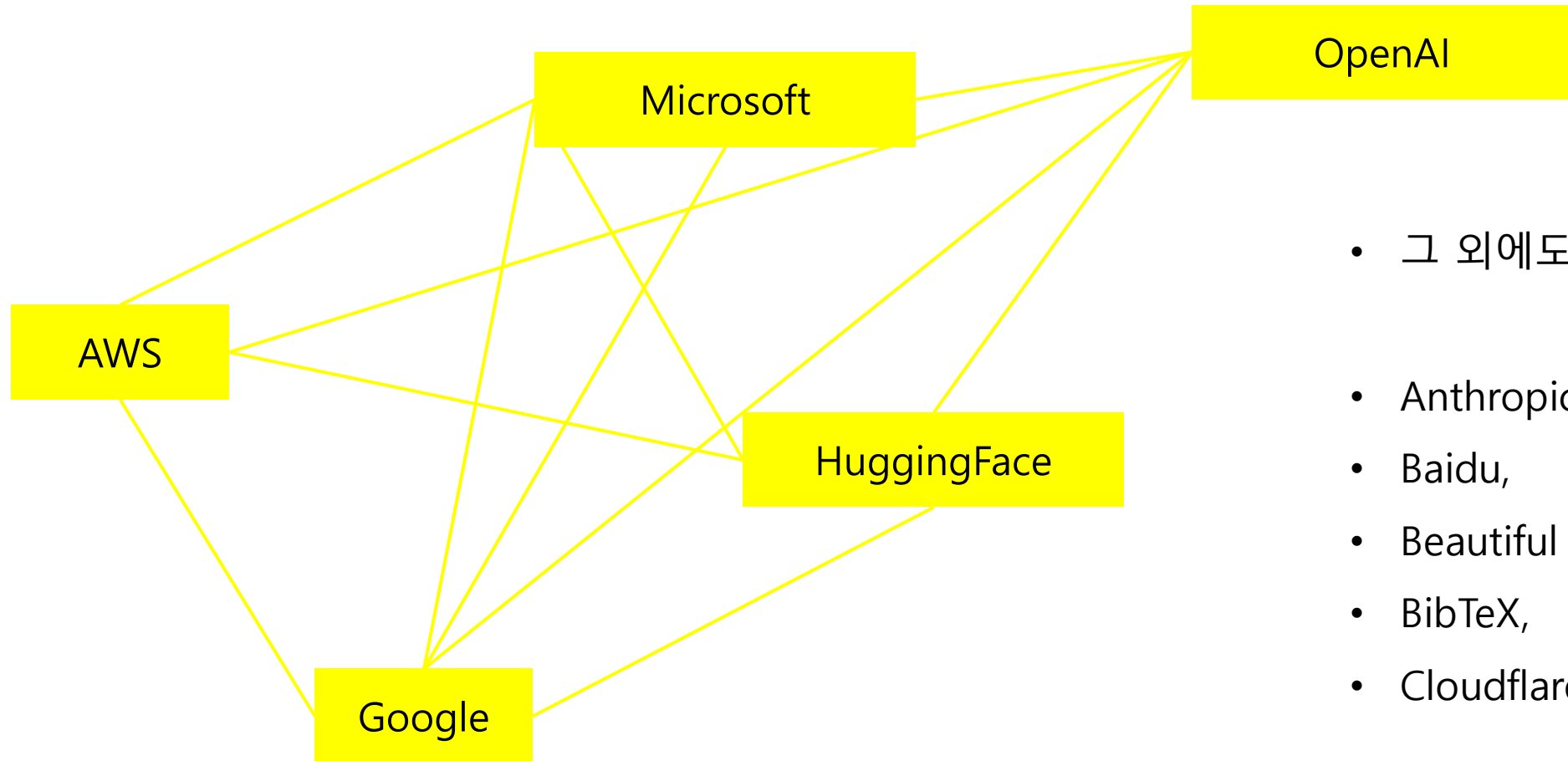
API(양식)에 맞추어 클라이언트가 내용을 보내면(요청), 서버도 API 양식에 맞춘 응답을 보냅니다(응답).



LangChain

3. LangChain의 구성요소 : Provider

- Provider : LangChain-{provider} 라는 패키지 명으로 접근 가능한, LLM 모델 제공사들을 말함



- 그 외에도
- Anthropic,
- Baidu,
- Beautiful Soup,
- BibTeX,
- Cloudflare 등

3. LangChain의 구성요소 : a. Components

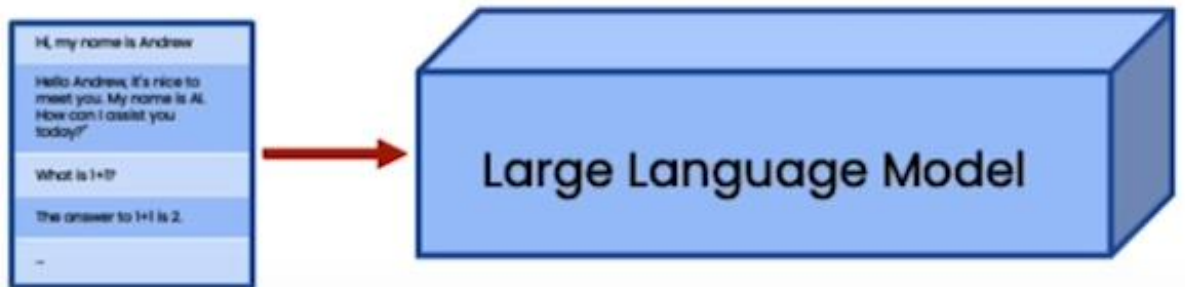


3. LangChain의 활용 : b. 모델, 프롬프트, 파서

- Lang Chain이 LLM과 정보를 주고받을 때 사용하는 API:
 - 프롬프트 : LLM 모델에 요청할 사항
 - 모델 : 사용할 LLM 모델
 - (output) 파서 : LLM 모델에 요청하여 받은 결과물을 특정한 포맷으로 변경하는 것

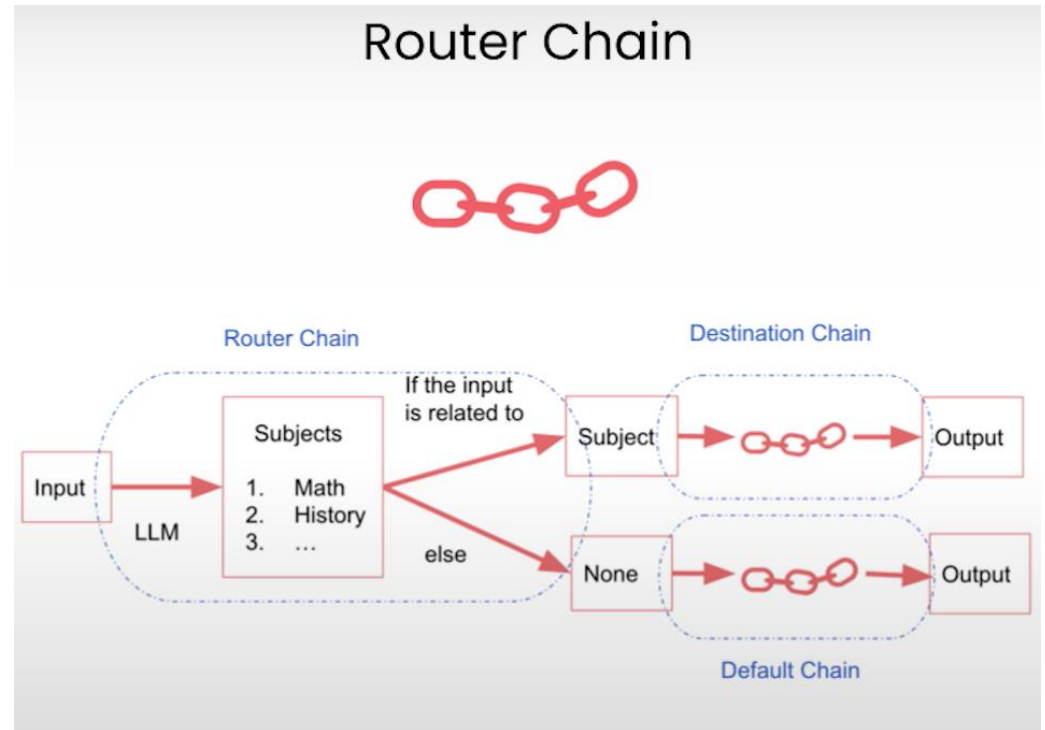
3. LangChain의 활용 : C. 메모리

- 실제로 LLM은 대화 내용을 기억하지는 않음(stateless)
 - 단, 이전 내용을 기억하는 것 처럼 '보이게' 하려고 이전 대화의 내용을 맥락으로 함께 제공
 - 이 때, LangChain 프레임워크는 이 '맥락' 을 기억하게 하는 몇 가지 메모리 모듈을 제공함
 - 실습 :
 - ConversationBufferMemory
 - ConversationBufferWindowMemory
 - ConversationTokenBufferMemory
 - ConversationSummaryMemory



3. LangChain의 활용 : D.체인

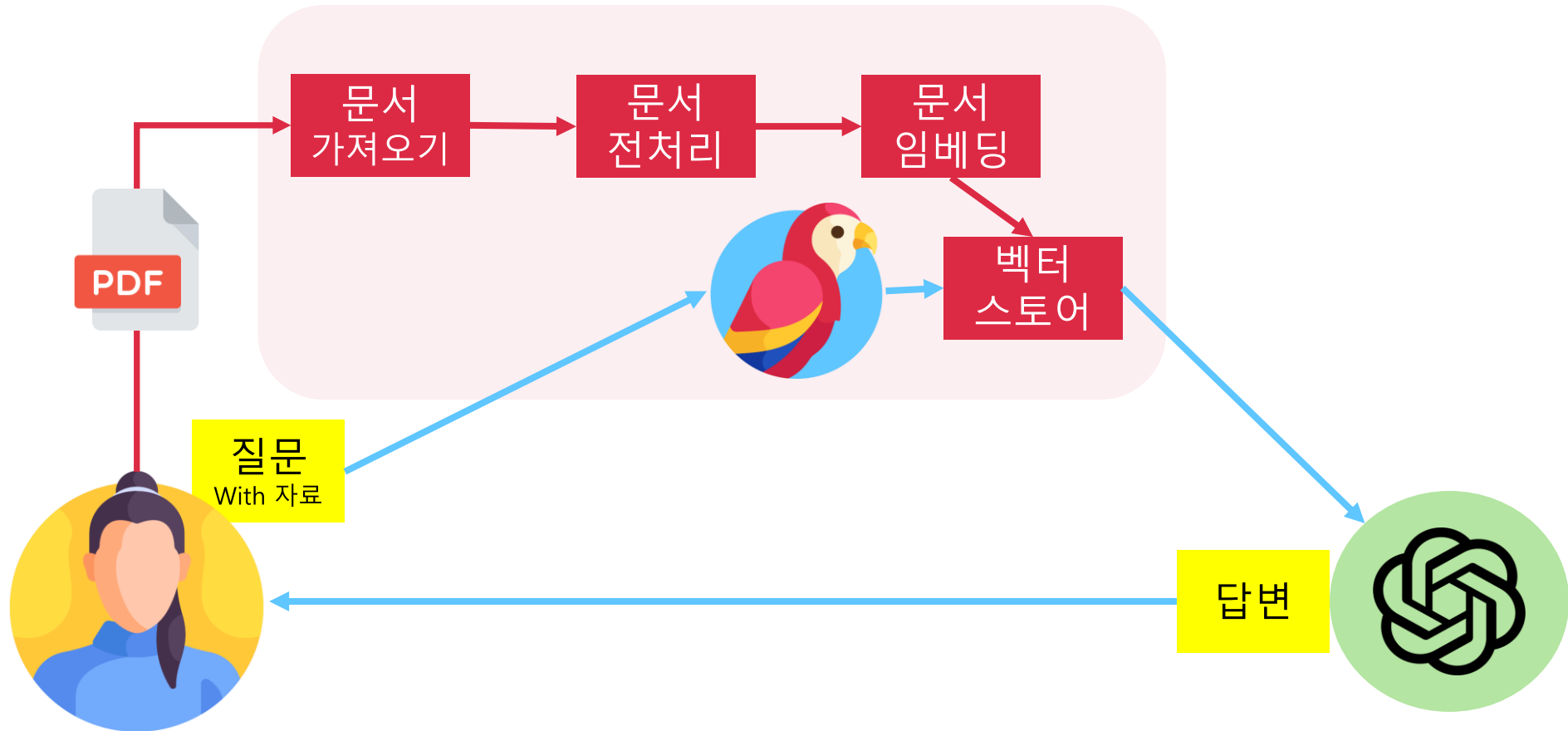
- LLM을 활용하여 질의-응답을 수행할 때, 여러 개의



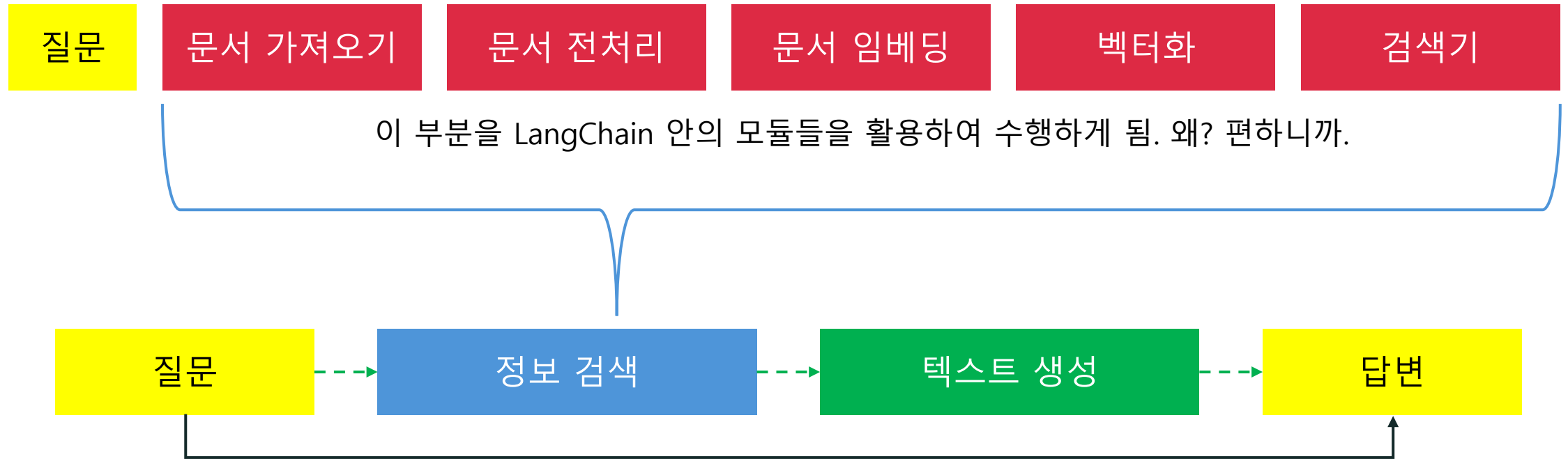
RAG

2: RAG(Retrieval-Augmented Generation)

- Retrieval (회수,검색) + Generation (생성)



2: RAG(Retrieval-Augmented Generation)



개발자(사용자)는 질문->답변의 순서로 동작한다고 인지하지만, 실제로는 점선(--) 으로 표시된 단계를 거쳐 답변을 받음

3. LangChain 튜토리얼 : a.데이터 검색

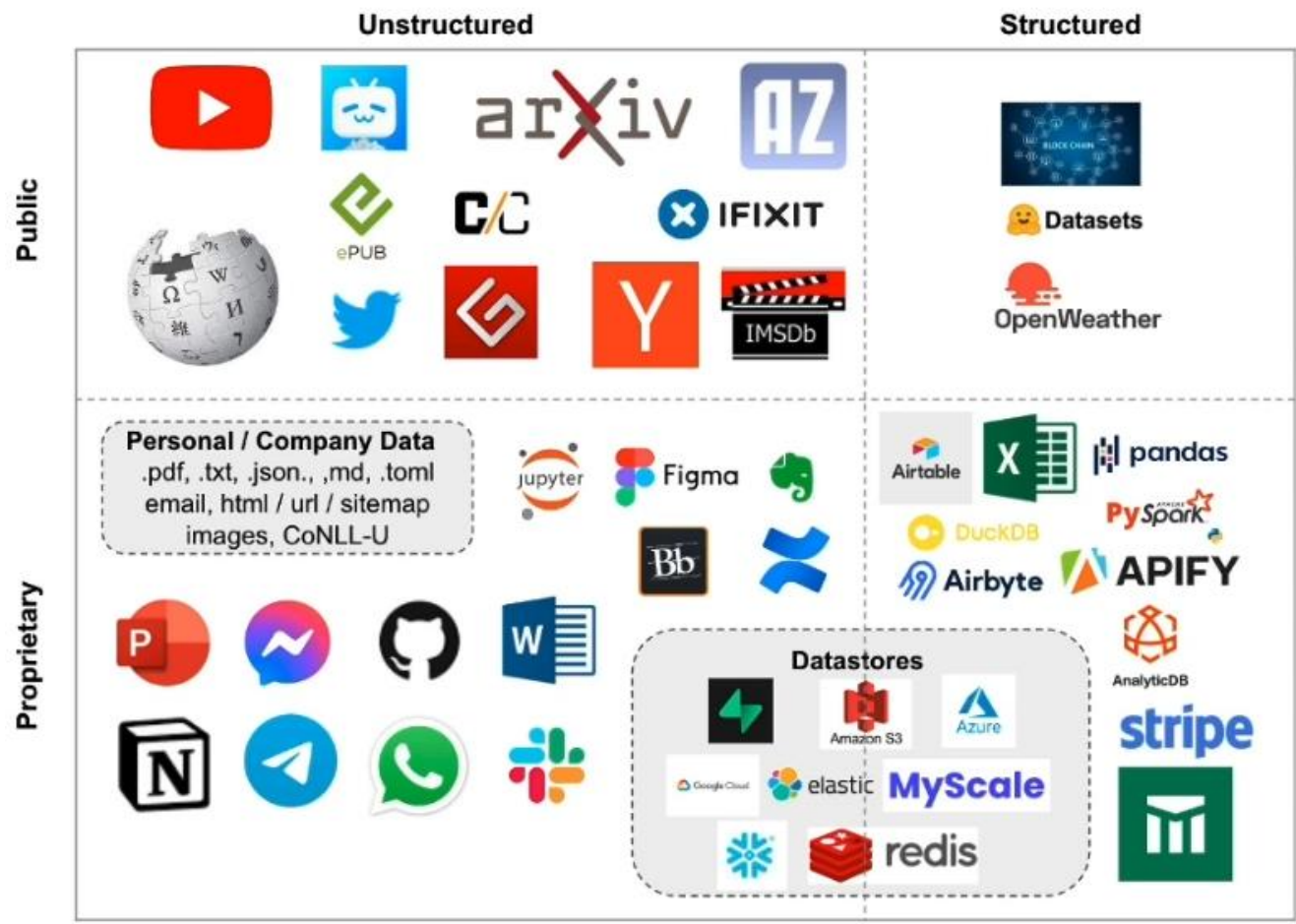
문서 가져오기

문서 전처리

문서 임베딩

벡터화

검색기



3. LangChain 튜토리얼 : a.데이터 검색

문서 가져오기

문서 전처리

문서 임베딩

벡터화

검색기

langchain.text_splitter.

- **CharacterTextSplitter()**- Implementation of splitting text that looks at characters.
- **MarkdownHeaderTextSplitter()** - Implementation of splitting markdown files based on specified headers.
- **TokenTextSplitter()** - Implementation of splitting text that looks at tokens.
- **SentenceTransformersTokenTextSplitter()** - Implementation of splitting text that looks at tokens.
- **RecursiveCharacterTextSplitter()** - Implementation of splitting text that looks at characters. Recursively tries to split by different characters to find one that works.
- **Language()** – for CPP, Python, Ruby, Markdown etc
- **NLTKTextSplitter()** - Implementation of splitting text that looks at sentences using NLTK (Natural Language Tool Kit)
- **SpacyTextSplitter()** - Implementation of splitting text that looks at sentences using Spacy

3. LangChain 튜토리얼 : a.데이터 검색

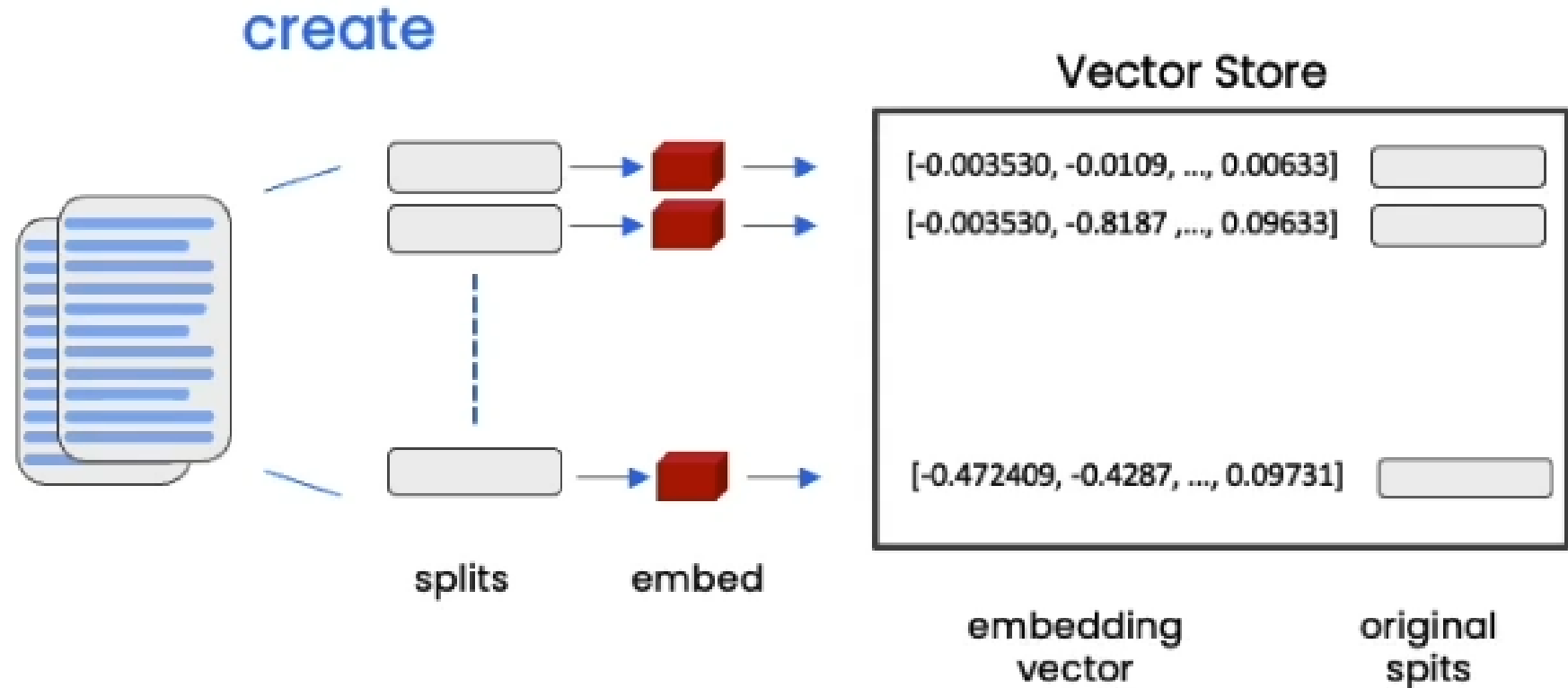
문서 가져오기

문서 전처리

문서 임베딩

벡터화

검색기



3. LangChain 튜토리얼 : a.데이터 검색

문서 가져오기

문서 전처리

문서 임베딩

벡터화

검색기

